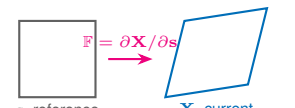
PK1 STRESS FUNCTIONS AND BODY FORCES

What IBAMR does with the first Piola–Kirchhoff stress, the neo-Hookean model in two parts, the other material laws in the examples, body and surface forces, and what the shear modulus and the tether stiffness do to the plate

$$\mathbb{P} = \mu\,\mathbb{F} + (-\mu + \beta \ln J)\,\mathbb{F}^{-T}$$



$$\mathbf{f}_k = -\int \mathbb{P}\,\nabla_{\mathbf{s}}\phi_k\,d\mathbf{s} + \int \mathbb{P}\mathbf{N}\,\phi_k\,dA$$

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 19

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. name the four quantities a stress function sees ($\mathbf{s}$, $\mathbf{X}$, $\mathbb{F}$, J) and the one it returns ($\mathbb{P}$, the first Piola–Kirchhoff stress), and say what IBAMR does with $\mathbb{P}$ at every quadrature point of every element at every time step;
2. write the neo-Hookean model as two registered functions, say why it is split, why $-\mu\mathbb{F}^{-T}$ is there (and that it does not "enforce incompressibility"), and what the quadrature orders do;
3. write the other models in IBAMR's examples: the compressible neo-Hookean solid with $\beta \ln J$ (IBFE/10, upstream ex4) and the St. Venant–Kirchhoff solid (upstream ex7), and choose their constants from E and ν;
4. give two parts of one mesh different materials by reference position or by subdomain;
5. write and register a body force (a tether, gravity, a damped tether that reads the velocity through `system_data`) and a surface force, and register them correctly: the 2020 slide that registered a body force as a stress does not compile;
6. say what the shear modulus and the tether stiffness do to the plate example, from runs: $\mu = 1, 10, 100$ and $\kappa_s = 10^4$, 10^6.

> The theory is in the theory repository (chapters 4.2 *Kinematics, strain, stress and hyperelasticity* and 4.3 *Strong form, weak form and discrete coupling*). This chapter is the code side: what to write, where IBAMR calls it, and what the numbers do.

WHERE THIS CHAPTER COMES FROM

- Alex Hoover, “3-IBFE Tutorial: PK1 Stress and Body Forces” (2014, edited 2016; 13 slides): the four quantities, the neo-Hookean deviatoric and dilational functions (with the pre-2016 seven-argument signature and `libMeshEnums::Order`), “stress components are additive”, the target-point body force, and the correct body-force registration. It refers to an `Example_IBFE_TargetPoints2D` that is in no archive; IBFE/01 is that example.
- “4-IBFE Lecture: Apply Active Tension” (2020, 23 slides; slides 1–14 are this chapter, 15–23 are chapter 20): the same with the current nine-argument signature, plus a slide (12) that registers the body force with `PK1StressFcnData` and `registerPK1StressFunction`, which is wrong (slide 19); the 2014 deck had it right.
- “2-IBFE Method: General Overview” (2018, 9 slides): four equation figures (probably cropped from Griffith & Luo; not reproduced), and the same three text slides on $\mathbf{s}$, $\mathbf{X}$, $\mathbb{F}$, $\mathbb{P}$ and the neo-Hookean parts.
- The three “Theory of IBFE” decks (*The formulation of IBFE*, *An Introduction to Tensors*, *Introduction to Neo Hookean Materials, PK1 stresses*; assigned in 2017) were never written; their titles are now the theory repository’s chapters 4.1–4.3 (153 slides).
- New here: the weak form IBAMR assembles (from `FEMechanicsBase.cpp`); the models of upstream ex4, ex7 and of IBFE/10 with μ and β from E and ν; two materials in one mesh; the velocity read through `system_data` (upstream ex5); surface forces on chosen sides (upstream ex8); gravity; the runs at $\mu = 1,\ 10,\ 100$ and $\kappa_s = 10^4$ with their pictures and numbers; the correction of “enforces incompressibility”.

THE QUANTITIES A STRESS FUNCTION SEES

symbol	in the code	meaning (the 2014 slide's words, corrected where marked)
$\mathbf{s}$	`s` (IBAMR: `X`)	"reference configuration of the element": where the material point started; what the mesh file holds
$\mathbf{X}$	`X` (IBAMR: `x`)	"current configuration of the element": where it is now
$\mathbb{F}$	`FF`	the deformation gradient, $F_{ij} = \partial X_i / \partial s_j$: "the derivative of each component of the deformed vector with respect to each component of the reference vector" (the 2014 and 2018 slides wrote this with the two names swapped, "deformed `x` ... reference `X`", IBAMR's convention)
J	`FF.det()`	the volume ratio, $J = \det \mathbb{F}$: 1 where nothing has expanded or shrunk
$\mathbb{P}$	`PP`	the first Piola–Kirchhoff stress: force per unit *reference* area, so that the force on a face is $\mathbb{P}\mathbf{N}\,dA$ with $\mathbf{N}$ and dA the reference normal and area; "expresses the stress relative to the reference configuration"
$\mathbb{F}^{-T}$	`tensor_inverse_transpose( FF, NDIM)`	inverse transpose; IBAMR's helper in `libmesh_utilities.h` (also `tensor_inverse`, `outer_product`)

THE QUANTITIES (2): WHERE THE THEORY IS, AND THE 2D READING

- Theory repository 4.2, modules 1–3: the deformation gradient, the Jacobian “as the product of principal stretches”, Cauchy–Green and Green–Lagrange strains, the polar decomposition, Cauchy, first and second Piola–Kirchhoff stresses, Nanson’s formula and the Piola transformation. The 2014 deck’s links were continuummechanics.org (deformation gradient), Wikipedia (Piola–Kirchhoff) and Bonet and Wood’s book (slide 33).
- In 2D, “area” reads “length” and “volume” reads “area” throughout: the plate’s $\mathbb{P}$ is force per unit reference length, its body force per unit reference area.

WHAT IBAMR DOES WITH $\mathbb{P}$

From `FEMechanicsBase.cpp` (the function that computes the interior elastic force density):

```
// Loop over the elements to compute the right-hand side vector.  This
// is computed via
//
//    rhs_k = -int{PP(s,t) grad phi_k(s)}ds + int{PP(s,t) N(s,t) phi_k(s)}dA(s)
//
// This right-hand side vector is used to solve for the nodal values of
// the interior elastic force density.
...
pk1.fcn(PP, FF, x, X, elem, PK1_var_data, PK1_grad_var_data, data_time, pk1.ctx);   // your function
...
F_qp = -PP * dphi[basis_n][qp] * JxW[qp];                                          // its contribution
```

- The structure's force density is the weak form of $\nabla_{\mathbf{s}} \cdot \mathbb{P}$: for every shape function ϕ_k, $\mathbf{f}_k = -\int \mathbb{P}\, \nabla_{\mathbf{s}} \phi_k \, d\mathbf{s} + \int \mathbb{P}\mathbf{N}\, \phi_k \, dA$. The nodal force is the mass-matrix solve of this right-hand side; it is then spread to the fluid from the quadrature points (chapter 17, slide 11).
- So your function is called at every quadrature point of every element, for every registered stress, at every time step. Keep it to arithmetic on `FF`; no file access, no allocation.
- Each registered stress has its own Gauss rule (`quad_type` `QGAUSS`, `quad_order` from the `*FcnData`; default `THIRD` when you give none). A higher order integrates a less smooth integrand better; that is the only reason the example asks `FIFTH` for $\mu\mathbb{F}$ and `THIRD` for $-\mu\mathbb{F}^{-T}$: slide 10.

WHAT IBAMR DOES WITH $\mathbb{P}$ (2): THE BOUNDARY TERM

- The boundary term $\int \mathbb{P}\mathbf{N}\phi_k \, dA$ is the traction on the mesh's boundary; with `split_forces = TRUE` IBAMR spreads it separately from the interior part, as a jump (the `use_jump_conditions` machinery; chapter 23).
- Theory repository 4.3: "Why IBFE uses a weak structural force", "Integration by parts retains the boundary term", "Mass-matrix projection of the weak force", "Quadrature-point force spreading".

THE NEO-HOOKEAN MODEL IN TWO FUNCTIONS

"For a neo-Hookean material model, the deviatoric component of stress is $\mathbb{P} = \mu\mathbb{F}$, where μ is our modulus of elasticity. Think of it as each node connected to two (in 2D) or three (in 3D) other nodes using a spring. The higher the μ, the stiffer the object" (2014). "We will also add a term for the dilational component, $\mathbb{P} = -\mu\mathbb{F}^{-T}$" (next slide for what it does).

```
// Stress tensor functions. The first Piola-Kirchhoff stress of the neo-Hookean
// material is split into two parts. The deviatoric part is mu*FF. The
// dilational part, -mu*FF^{-T}, makes the undeformed plate stress-free: at
// FF = I the two parts cancel.
void PK1_dev_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, const libMesh::Point& /*X*/,
                             const libMesh::Point& /*s*/, Elem* const /*elem*/,
                             const std::vector<const std::vector<double>*>& /*var_data*/,
                             const std::vector<const std::vector<VectorValue<double> >*>& /*grad_var_data*/,
                             double /*time*/, void* /*ctx*/)
{
    PP = mu * FF;
}
void PK1_dil_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, ...the same arguments...)
{
    PP = -mu * tensor_inverse_transpose(FF, NDIM);
}
```

- μ is `ModelData::mu`, set from `SHEAR_MODULUS` in the input (chapter 17). It is a *shear* modulus: the neo-Hookean energy is $W = \frac{\mu}{2}(I_1 - 3)$ with $I_1 = \operatorname{tr}\mathbb{F}^T\mathbb{F}$, and $\partial W/\partial\mathbb{F} = \mu\mathbb{F}$ (theory repository 4.3, "Differentiating the neo-Hookean energy").
- "Think springs": under a uniform stretch λ along x, $\mathbb{F} = \operatorname{diag}(\lambda, 1)$ and $\mu\mathbb{F}$ pulls back with $\mu\lambda$: linear in the stretch, like a spring with constant μ per unit reference length. Chapter 8b's spring constant k and this μ play the same role.

WHY $-\mu\mathbb{F}^{-T}$ IS THERE, AND WHAT IT DOES NOT DO

- With $\mathbb{P} = \mu\mathbb{F}$ alone, the undeformed plate ($\mathbb{F} = \mathbb{I}$) carries the stress $\mu\mathbb{I}$: a uniform tension that would pull every boundary inward from the first step. The term $-\mu\mathbb{F}^{-T}$ equals $-\mu\mathbb{I}$ at $\mathbb{F} = \mathbb{I}$ and cancels it: the reference state is stress-free.
- Away from $\mathbb{F} = \mathbb{I}$ the two do not cancel: for the stretch $\mathrm{diag}(\lambda, 1)$ (2D), $\mathbb{P} = \mu\,\mathrm{diag}(\lambda - 1/\lambda,\ 0)$: tension along the stretch, nothing across, and zero at $\lambda = 1$.
- The 2014, 2018 and 2020 slides say the term "enforces the incompressibility condition". It does not. In IBFE the solid moves with the fluid's velocity, and the fluid is incompressible ($\nabla \cdot \mathbf{u} = 0$), so the solid's volume is conserved by the fluid solver, not by the stress; J stays near 1 for that reason (theory repository 4.3, "Incompressibility of the immersed material"). $-\mu\mathbb{F}^{-T}$ is the pressure-like part of the stress, $-p\mathbb{F}^{-T}$ with $p = \mu$, chosen to make the reference state stress-free.
- Where the solid *can* change volume (coarse quadrature, large time steps, a body thinner than a grid cell), J drifts; the compressible models of slide 12 add $\beta \ln J$ to push it back, and IBFE/10 prints the range of J it sees every 50 steps (`J from 0.82 to 1.73` in the flow-tank run, chapter 22).
- The Wikipedia page the slides linked (*Neo-Hookean solid*, incompressible case) has the Cauchy stress $\boldsymbol{\sigma} = \mu\mathbb{B} - p\mathbb{I}$ with p a Lagrange multiplier; $\mathbb{P} = J\boldsymbol{\sigma}\mathbb{F}^{-T} = \mu\mathbb{F} - p\mathbb{F}^{-T}$ at $J = 1$, which is the two-part form with $p = \mu$.

> Keep both parts. Drop the dilational one and the plate shrinks from $t = 0$; drop the deviatoric one and it has no stiffness at all.

WHY TWO FUNCTIONS: “STRESS COMPONENTS ARE ADDITIVE”

- “Stress components are additive, so all we need to do is register them in the main routine” (2014). IBAMR sums every registered stress at each quadrature point, so a material law can be written as a list of terms, each with its own quadrature rule.
- $\mu\mathbb{F}$ is as smooth as $\mathbb{F}$: on a linear element $\mathbb{F}$ is constant, and any rule integrates it exactly; the course still asks `FIFTH`. $\mathbb{F}^{-T}$ is a rational function of the nodal positions, less smooth on a quadratic element, so one might expect it to want the higher order; the course gives it `THIRD`. In practice both work with `THIRD`: IBAMR’s own ex2 uses `FIFTH`/`THIRD` as the course does, ex5 and ex9 `SEVENTH`/`FIFTH` with quadratic elements, ex2 and ex4 in 3D `THIRD`/`FIRST`.
- The split also lets an *active* stress (chapter 20) be a third registered function, switched on and off in time, without touching the passive law.

```
IBFEMethod::PK1StressFcnData PK1_dev_stress_data(PK1_dev_stress_function);
IBFEMethod::PK1StressFcnData PK1_dil_stress_data(PK1_dil_stress_function);
PK1_dev_stress_data.quad_order = Utility::string_to_enum<libMesh::Order>(input_db->getStringWithDefault("PK1_DEV_QUAD_ORDER", "THIRD
  "));
PK1_dil_stress_data.quad_order = Utility::string_to_enum<libMesh::Order>(input_db->getStringWithDefault("PK1_DIL_QUAD_ORDER", "FIRST
  "));
ib_method_ops->registerPK1StressFunction(PK1_dev_stress_data);
ib_method_ops->registerPK1StressFunction(PK1_dil_stress_data);
```

- The 2014 version read `Utility::string_to_enum<libMeshEnums::Order>`; the enum moved into `libMesh::` in 2016. The `quad_type` field stays at its default (`QGAUSS`).
- A `PK1StressFcnData` with no `quad_order` takes the part’s default, `THIRD`.

THE 2014 SIGNATURE AND TODAY'S

2014 (seven arguments; "`mu = 100`"):

```
static double mu = 100;
void PK1_dev_stress_function(
    TensorValue<double>& PP,
    const TensorValue<double>& FF,
    const libMesh::Point& X,
    const libMesh::Point& s,
    Elem* const /*elem*/,
    const vector<NumericVector<double>*>& /*system_data*/,
    double time,
    void* /*ctx*/)
{
PP=mu*FF;
return;
}
```

2016 and later (nine arguments; chapter 17, slide 14):

```
void PK1_dev_stress_function(
    TensorValue<double>& PP,
    const TensorValue<double>& FF,
    const libMesh::Point& /*X*/,
    const libMesh::Point& /*s*/,
    Elem* const /*elem*/,
    const std::vector<const std::vector<double>*>& /*var_data*/
  ,
    const std::vector<const std::vector<VectorValue<double>
  >*>& /*grad_var_data*/,
    double /*time*/,
    void* /*ctx*/)
{
    PP = mu * FF;
}
```

- The one `system_data` vector of raw `NumericVectors` became two vectors of element-local values and gradients (`var_data`, `grad_var_data`), filled from the `SystemData` list you give the `*FcnData` (slide 22).
- A function with the 2014 signature fails to compile against 0.16.0 and later ("no matching function for call to `IBFEMethod::PK1StressFcnData::PK1StressFcnData(void (&)(...))`"): the error of chapter X3's table.
- The 2014 slide's `mu = 100` versus the example's 10: the two course examples used different plates. Slide 16 shows what the factor ten does.

OTHER MODELS IN IBAMR'S EXAMPLES (1): COMPRESSIBLE NEO-HOOKEAN

Upstream `IBFE/explicit/ex4` (a disc or ball with `P0_S`, `BETA_S`, `c1_s = 0.05`):

```
PP = 2.0 * c1_s * FF;                                                    // deviatoric
PP = 2.0 * (-p0_s + beta_s * log(FF.det())) * tensor_inverse_transpose(FF, NDIM);  // dilational
```

Example IBFE/10 (the flow-tank plate, chapter 22), the same law written with μ and β:

```
PP = mu * FF;                                                             // deviatoric
const double J = FF.det();
PP = (-1.0 * mu + beta * std::log(J)) * tensor_inverse_transpose(FF, NDIM);       // dilational
```

- $\mathbb{P} = \mu\mathbb{F} + (-\mu + \beta \ln J)\mathbb{F}^{-T}$: at $\mathbb{F} = \mathbb{I}$, $J = 1$ and the stress is zero; a volume change gives $\beta \ln J\, \mathbb{F}^{-T}$, a pressure that opposes it. β is the first Lamé parameter λ; $c_1 = \mu/2$ in ex4's notation (its `p0_s` is a reference pressure, μ in the plate's).
- From Young's modulus E and Poisson's ratio ν: $\mu = E/(2(1+\nu))$ and $\beta = E\nu/((1+\nu)(1-2\nu))$. IBFE/10's input does this arithmetic (`E = 1.4`, $\nu = 0.3$) and inherits a slip from 2017: the two μ lines use each other's E (`MU_STEM` from `E_LEAF`), so the plate runs with $\mu_s = 0.538$ and $\beta = 0.404$; the README keeps it, documented, so the runs match the course's.
- $\nu \to 0.5$ sends $\beta \to \infty$: a nearly incompressible solid is a stiff penalty, and a stiff penalty is a small time step. $\nu = 0.3$–0.4 is the usual compromise; the fluid keeps the volume anyway (slide 9).
- Theory repository 4.3: "Compressible neo-Hookean energy and parameters", "Stress response of the homogeneous stretch", and the verification checklist built on it.

OTHER MODELS (2): ST. VENANT–KIRCHHOFF, AND A STRESS THAT DEPENDS ON WHERE YOU ARE

Upstream `IBFE/explicit/ex7` (a channel whose upper wall is elastic between $x = 5$ and 10 and tethered elsewhere; chapter 23 walks through the file):

```
static double mu_s, lambda_s;
void upper_PK1_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, const libMesh::Point& /*X*/,
                               const libMesh::Point& s, Elem* const /*elem*/, ...var_data..., double /*time*/, void* /*ctx*/)
{
    if (s(0) > 5.0 && s(0) < 10.0)
    {
        static const TensorValue<double> II(1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0);
        const TensorValue<double> CC = FF.transpose() * FF;              // right Cauchy-Green
        const TensorValue<double> EE = 0.5 * (CC - II);                  // Green-Lagrange strain
        const TensorValue<double> SS = lambda_s * EE.tr() * II + 2.0 * mu_s * EE;   // second Piola-Kirchhoff stress
        PP = FF * SS;                                                    // first = F times second
    }
    else
    {
        PP.zero();                                                       // the tethered parts: no elasticity
    }
}
```

OTHER MODELS (3): WHAT THE ST. VENANT–KIRCHHOFF CODE SAYS, AND "WHICH MATERIAL"

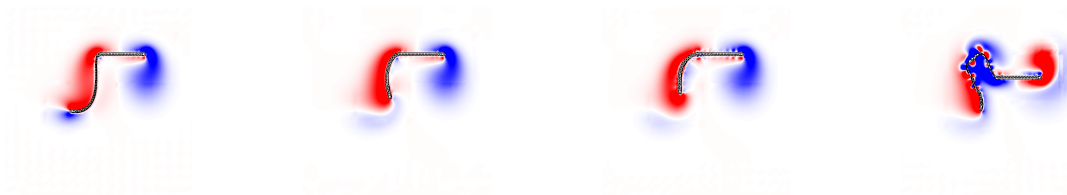
- St. Venant–Kirchhoff: Hooke's law written in $\mathbb{E}$ and $\mathbb{S}$, with λ and μ the Lamé parameters; $\mathbb{P} = \mathbb{F}\mathbb{S}$ (theory repository 4.2, "Second Piola–Kirchhoff stress", "Piola transformation"). Simple, but soft under compression: for large strains the neo-Hookean law is the safer default.
- Position-dependent material: `s(0)` is the reference x; the test $5 < s(0) < 10$ picks the elastic part of the wall. IBFE/10 does the same with $|s(2)| \geq 0.25$ to tell a stiff "stem" from a soft "leaf" (`TWO_MATERIALS`). The reference position never changes, so the test is cheap and stable.
- The other way to say "which material" is `elem->subdomain_id()`, set by the mesh's physical groups or Exodus blocks (chapter 18); chapter 21 uses it.

THE MODELS SIDE BY SIDE

model	$\mathbb{P}$	constants	where
neo-Hookean, two parts	$\mu\mathbb{F} - \mu\mathbb{F}^{-T}$	μ (shear modulus)	IBFE/01, 05, 06, 08, 12; upstream ex2, ex3, ex6, ex8; the course's plates
compressible neo-Hookean	$\mu\mathbb{F} + (-\mu + \beta \ln J)\mathbb{F}^{-T}$	μ, $\beta = \lambda$; from E, ν	IBFE/10; upstream ex4 ($2c_1\mathbb{F} + 2(-p_0 + \beta \ln J)\mathbb{F}^{-T}$), ex5, ex9
St. Venant–Kirchhoff	$\mathbb{F}\,(\lambda \operatorname{tr}\mathbb{E}\,\mathbb{I} + 2\mu\mathbb{E})$, $\mathbb{E} = \frac{1}{2}(\mathbb{F}^T\mathbb{F} - \mathbb{I})$	λ, μ	upstream ex7
active fibre stress	$J\,T\,\mathbb{F}\,\mathbf{f}_0 \otimes \mathbf{f}_0$ added to a passive law	$T(\mathbf{s}, t)$, fibre direction $\mathbf{f}_0$	chapter 20 (IBFE/06)

- All are hyperelastic: $\mathbb{P} = \partial W / \partial \mathbb{F}$ for an energy $W(\mathbb{F})$; the theory repository's 4.3 derives the neo-Hookean one and tests it on a homogeneous stretch ("Kinematic test and fully coupled IBFE simulation"). Any other law from the literature (Mooney–Rivlin, Fung, Holzapfel–Ogden for tissue) is written the same way: compute $\mathbb{P}$ from $\mathbb{F}$ in one function, split it into parts if you want separate quadrature, register the parts.
- Write the law in terms of `FF` only when you can; `X`, `s`, `elem` and `time` are there for position-dependent and time-dependent parts, and `var_data` for fields stored on the mesh (slide 22).

WHAT THE SHEAR MODULUS DOES: THE HALF-TETHERED PLATE AT $t = 1$



$\mu = 1, t = 1$ $\mu = 10$ (shipped), $t = 1$ $\mu = 100, t = 1$ $\mu = 1, t = 1.5$: crumpled

- IBFE/01 with `MOTION = "HALF"`: the right half is pulled up at 0.5 for one second, the left half is carried by the material law alone. Vorticity ± 5, the mesh displaced by `dX`.
- $\mu = 100$: the free half follows almost rigidly, the plate tilts by about $32°$. $\mu = 10$: about $39°$, with a visible bend at the join. $\mu = 1$: the free half folds under the fluid's drag, its tip moves from $x = -0.5$ to about -0.15 by $t = 1.5$, the folding plate stirs up fast flow, and the run stops near $t = 1.6$ with `Time step size change encountered`; halving `DT` only delays it to $t \approx 2.05$ (chapter 16).
- A factor of ten in μ is a factor of ten in the restoring force for the same strain; the drag on the free half is fixed by the motion, so the strain scales as $1/\mu$ until it is no longer small. Chapter 8b's non-dimensional stiffness: $\mu/(\rho U^2)$ with $U = 0.5$ and $\rho = 1$ is 4, 40 and 400 here.

Runs: `/root/work/0.19.0/try_softplate_dt` ($\mu = 1$, `DT = 5e-4`), `ibfe01_HALF`, `try_stiffplate`; IBAMR 0.19.0; `wsl/final/visit_ch19.sh`.

CHOOSING μ

- Units. $\mathbb{P}$ is force per reference area (2D: per reference length), so μ has the units of a pressure (2D: force per length). In the examples' unit system ($\rho = 1$, $L = 2$, $U = 0.5$) a value is a multiple of ρU^2; chapter 8b's rule applies: fix the dimensionless group, not the number.
- From a real material: $\mu = E/(2(1+\nu))$; soft tissue $E \sim 10^4$–10^6 Pa, rubber $\sim 10^6$ Pa, with $\nu \approx 0.45$–0.5. Convert to the simulation's units through ρU^2 (a plate in water, $\rho = 1000$ kg/m^3, $U = 0.1$ m/s: $\rho U^2 = 10$ Pa, so $E = 10^4$ Pa is $\mu/(\rho U^2) \approx 350$).
- From the behaviour you want: the plate's bending stiffness goes as μh^3 per unit width (h the thickness); halve h and you need eight times μ for the same deflection. The IBFE/01 runs give the scale: $\mu/(\rho U^2) = 40$ bends visibly under its own motion, 4 folds, 400 hardly bends.
- The cost of stiffness: the explicit coupling limits the time step by the elastic wave speed, $\Delta t \lesssim \mathrm{DX}/\sqrt{\mu/\rho}$ up to a constant; $\mu = 100$ ran at `DT = 1e-3` ($\sqrt{\mu/\rho}\,\Delta t/\mathrm{DX} = 0.64$), and the stiff-tether chapter 9a rule "stiffness against `DT`" holds here too. A plate that must not bend at all is better tethered (chapter 17) than made stiff.
- The cost of softness: large strains, elements that fold, and a run that ends in `Time step size change encountered` when the folding body drives the fluid past the CFL limit.

> Start from the example's μ, predict the change you expect from $\mu/(\rho U^2)$, run, and compare with the measured tilt: chapter 8a's procedure, which is why chapter 17 recorded the numbers.

BODY FORCES: THE SIGNATURE, THE TETHER, AND THE 2014 REMARK ABOUT $\mathbf{F}$

"Similar to PK1 stress, you are applying a generalized force directly on the element. This is a lot easier than in the fiber version of IBAMR because you can use positional information from the reference configuration, `s`. Also, because it is generalized, the forcing function can be anything of your choosing" (2014). A body force is a vector per unit reference volume (2D: area), returned in `F`, from the same nine arguments:

```
void body_force_function(VectorValue<double>& F, const TensorValue<double>& FF, const libMesh::Point& X, const libMesh::Point& s,
                         Elem* const elem, const std::vector<const std::vector<double>*>& var_data,
                         const std::vector<const std::vector<VectorValue<double> >*>& grad_var_data, double time, void* ctx);
```

The 2014 and 2020 target-point force (chapter 17's tether, in the course's spelling):

```
{  libMesh::Point s_dump;
   kappa_s=1e6; //Stiffness constant
    if(time<1)      { s_dump(1)=s(1)+.5*time; s_dump(0)=s(0); }      // up at 0.5 for 0 < t < 1
    else if(time<3) { s_dump(1)=s(1)+.5+.5*(1-time); s_dump(0)=s(0); }
    else            { s_dump(1)=X(1); s_dump(0)=X(0); }              // X(.): current configuration: no force
    F = kappa_s*(s_dump-X);      //usual target point movement
    return;
}
```

- "Note that for our body force function, we want to update $\mathbf{F}$ at the end of our function" (2014): `F` is the output argument; whatever it holds when the function returns is the force. Set it on every path (the `s_dump = X` branch gives zero; `F.zero()` is the explicit form).
- κ_s multiplies a displacement, so it is a stiffness per unit reference volume: force/length$^{\mathrm{NDIM}+1}$. 10^6 in the plate's units; `KAPPA_S` in the input.

REGISTERING A BODY FORCE, AND THE SLIDE THAT GOT IT WRONG

Correct (2014 deck, slide 12; the example):

```
IBFEMethod::LagBodyForceFcnData body_force_data(
  body_force_function);
// the second argument is the part (default 0)
ib_method_ops->registerLagBodyForceFunction(body_force_data, 0);
```

Wrong (2020 deck, slide 12, under the heading “Registering your Body Force”):

```
IBFEMethod::PK1StressFcnData active_stress_data(
  active_stress_function);
ib_method_ops->registerPK1StressFunction(active_stress_data);
```

REGISTERING A BODY FORCE (2): WHAT WENT WRONG, AND THE COUNTING RULES

- The 2020 slide pasted the active-*stress* registration (chapter 20) under the body-force heading. A body force returns a `VectorValue`; a stress returns a `TensorValue`; the two `*FcnData` types take different function pointers, so putting a body force into `PK1StressFcnData` does not compile. Nobody could have run it, which is how it survived four years.
- One body force function per part (a second `registerLagBodyForceFunction` replaces the first); any number of stress functions; one surface force and one surface pressure per part (chapter 17, slide 25). A plate with a tether *and* gravity needs both in one function.
- The second argument is the part: ex7 registers `lower_tether_force_data` on part 0 and `upper_tether_force_data` on part 1.
- "When done remember to register your equation system" (2014 slide 13): `initializeFEEquationSystems()` after the registrations, then `getFEDataManager(part)->getEquationSystems()` for the writer; chapter 17, slide 23.

BODY FORCES (2): GRAVITY, AND ANYTHING ELSE "OF YOUR CHOOSING"

A body that is heavier than the fluid: gravity minus buoyancy, per unit reference volume, on every point:

```
static double rho_s = 1.2, rho_f = 1.0, g = 9.81;      // from the input: RHO_S, RHO (and g in the simulation's units)
void gravity_force_function(VectorValue<double>& F, const TensorValue<double>& /*FF*/, const libMesh::Point& /*X*/,
                            const libMesh::Point& /*s*/, Elem* const /*elem*/,
                            const std::vector<const std::vector<double>*>& /*var_data*/,
                            const std::vector<const std::vector<VectorValue<double> >*>& /*grad_var_data*/,
                            double /*time*/, void* /*ctx*/)
{
    F.zero();
    F(1) = -(rho_s - rho_f) * g;     // down; the fluid's own weight is in the pressure already
}
```

- The IBFE solid has the fluid's density (it *is* fluid, constrained to move elastically): a body force $(\rho_s - \rho_f)\mathbf{g}$ is how excess weight enters, the counterpart of chapter 13's point masses. Buoyancy is already in the fluid's hydrostatic pressure, hence the difference.
- A tether plus gravity: add the two terms in one function. A force that switches with time: test `time`, as the tether does. A force on part of the body: test `s`, as IBFE/10's band does (`|s(2)| < 0.1`). A force that depends on the deformation: use `FF` (a damper proportional to the rate needs the velocity, next slide).
- Chapter 20's active tension is *not* a body force but a stress: it acts along fibres and is integrated by parts like $\mathbb{P}$. The theory repository's 4.3 has both: "Localized tether as an external body force" and "Passive and active stress".

READING A FIELD FROM THE MESH: SYSTEM_DATA, AND A DAMPED TETHER (UPSTREAM EX5)

`var_data` and `grad_var_data` are empty unless you ask for systems. Upstream `IBFE/explicit/ex5` asks for the velocity so that its tether can damp:

```
struct TetherData { double kappa_s_body, eta_s_body, kappa_s_surface, eta_s_surface; TetherData(Pointer<Database> input_db) {...} };
void tether_force_function(VectorValue<double>& F, const TensorValue<double>& /*FF*/, const libMesh::Point& x, const libMesh::Point&
   X,
                           Elem* const /*elem*/, const vector<const vector<double>*>& var_data,
                           const vector<const vector<VectorValue<double>>*>& /*grad_var_data*/, double /*time*/, void* ctx)
{
    const TetherData* const tether_data = reinterpret_cast<TetherData*>(ctx);
    const std::vector<double>& U = *var_data[0];                          // the velocity at this quadrature point
    for (unsigned int d = 0; d < NDIM; ++d)
        F(d) = tether_data->kappa_s_body * (X(d) - x(d)) - tether_data->eta_s_body * U[d];   // spring minus damper
}
...
std::vector<int> vars(NDIM); for (unsigned int d = 0; d < NDIM; ++d) vars[d] = d;
vector<SystemData> sys_data(1, SystemData(IBFEMethod::VELOCITY_SYSTEM_NAME, vars));
IBFEMethod::LagBodyForceFcnData body_fcn_data(tether_force_function, sys_data, &tether_data);
ib_method_ops->registerLagBodyForceFunction(body_fcn_data);
```

READING A FIELD FROM THE MESH (2): WHAT SYSTEMDATA AND CTX DO

- `SystemData(system_name, vars, grad_vars)` names a libMesh system on the mesh and which of its variables you want; IBAMR evaluates them at each quadrature point and hands them over as `var_data[i]` (values) and `grad_var_data[i]` (gradients), in the order you listed. `IBFEMethod::VELOCITY_SYSTEM_NAME` is the structure's velocity; `COORDS_SYSTEM_NAME` its position; chapter 20 adds a system of its own for the fibre direction.
- `ctx` carries your parameters without globals: a pointer to any struct, cast back inside. The course used `static` variables in `ModelData`; both work, `ctx` scales to several parts with different constants.
- Note the naming: ex5 follows IBAMR's headers (`x` current, `X` reference), the reverse of the course's.

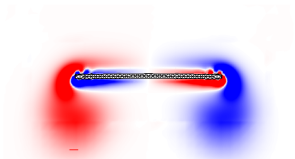
SURFACE FORCES: A TRACTION ON CHOSEN SIDES (UPSTREAM EX8)

```
static double beam_kappa_s = 1.0e6;
void beam_tether_force_function(VectorValue<double>& F,
                                const VectorValue<double>& /*n*/, const VectorValue<double>& /*N*/,
                                const TensorValue<double>& /*FF*/,
                                const libMesh::Point& X, const libMesh::Point& s,
                                Elem* const /*elem*/, const unsigned short int side,
                                const vector<const vector<double>*>& /*var_data*/,
                                const vector<const vector<VectorValue<double>>*>& /*grad_var_data*/,
                                double /*time*/, void* /*ctx*/)
{
    if (side == 1 || side == 3)             // the two long sides of the beam's quadrilaterals
    {
        F = beam_kappa_s * (s - X);         // a tether applied on the surface, per unit reference area
    }
    else
    {
        F.zero();
    }
}
...
// part 2 of three
ib_method_ops->registerLagSurfaceForceFunction(beam_tether_force_data, 2);
```

SURFACE FORCES (2): SIDES, UNITS, AND WHERE THEY ARE USED

- Twelve arguments (chapter 17, slide 26): the nine plus the current and reference normals `n`, `N` and the element's `side`. libMesh numbers a quadrilateral's sides 0–3 counter-clockwise from the bottom edge and a triangle's 0–2; `side` lets one function act on the long edges only.
- A surface force is per unit reference area of the boundary; it enters the weak form as $\int \mathbf{F}\phi_k \, dA$ on the boundary, alongside the traction $\mathbb{P}\mathbf{N}$. Upstream ex5 registers the *same* damped tether as a body force on the interior and as a surface force on the boundary of a thin shell; the immersed-interface examples (`IIM/ex0`–`ex6`) use surface forces for everything.
- A surface *pressure* (scalar; force $-p\,\mathbf{n}$) is the special case for loads normal to the surface: chapter 17, slide 27.

WHAT THE TETHER STIFFNESS DOES: THE FULL PLATE WITH $\kappa_s = 10^4$



$\kappa_s = 10^4$, $t = 1$

- With the shipped $\kappa_s = 10^6$ the plate's centre is at $y = 0.500$ at $t = 1$, the target's position to three digits (chapter 17, slide 38).
- With $\kappa_s = 10^4$ it trails: $y = 0.241$ instead of 0.250 at $t = 0.5$, a lag of 0.009, and when the target turns around at $t = 1$ the plate overshoots to 0.507. The lag is the drag divided by κ_s times the plate's area: a hundred times softer, a hundred times the lag (10^{-4} at 10^6).
- At 10^2 the plate would lag by about 1, its own length: it no longer follows at all, and the run becomes "a plate in a fluid with a weak spring". Chapter 9a's rule for target points, κ large enough that the lag is below the grid spacing, is the same rule: $\kappa_s \gtrsim F_{\text{drag}}/(A\,\text{DX})$.
- The stiffness-against-DT rule also carries over: $\kappa_s = 10^6$ at `DT = 1e-3` is near the limit for this grid (a stiffer tether at the same step stops with the chapter 16 error).

Run: `/root/work/0.19.0/try_softtether` (`MOTION = "FULL"`, `KAPPA_S = 1.0e4`), IBAMR 0.19.0.

WHAT CAN GO WRONG

symptom	cause and fix
`no matching function for call to ...`	the function's signature is not the nine-argument one (a 2014 seven-argument function; a body force given to a stress struct, as on the 2020 slide); fix the arguments, use the matching struct
`PK1StressFcnData(void (&)(...))` or `LagBodyForceFcnData(...)`	
the plate shrinks or pulls in from $t = 0$	the dilational part $-\mu \mathbb{F}^{-T}$ is missing or has the wrong sign; the reference state must give $\mathbb{P} = 0$
the body has no stiffness, folds at once	the deviatoric part is missing; or μ is in the wrong units (slide 17)
`nan` in the log, then an abort	$\ln J$ of $J \leq 0$: an inverted element (chapter 18's orientation rule), or a crumpled one (too soft, or a step too large)
`Time step size change encountered`	the body is too soft for its motion (folding drives the flow past CFL), or too stiff for `DT`; chapter 16
the tether does not hold	κ_s too small for the drag (slide 26); or the force was set on one branch only ("update `F` at the end")
the force acts on the wrong part	the part argument of `register*` (default 0); or `s` tested in the wrong coordinate (`s(1)` is y)
a second body force silently replaces the first	one body force per part: combine them in one function
the run is slow in the structure step	work inside the stress function: allocation, I/O, or `std::pow` on every call; precompute, use `ctx`
`var_data[0]` is empty or out of range	no `SystemData` given to the `*FcnData`, or fewer systems than you index

WHAT THE OLDER DECKS HAD THAT IS NOW PART X

2014–2020	now
seven-argument callbacks with `const vector<NumericVector<double>*>& system_data`	nine arguments with `var_data` and `grad_var_data` (slide 11)
`Utility::string_to_enum< libMeshEnums::Order>`	`libMesh::Order`
“`-mu*F^-T` enforces the incompressibility condition” (2014, 2018, 2020)	it makes the reference state stress-free; the fluid keeps the volume (slide 9)
2020 slide 12: a body force registered with `PK1StressFcnData`	`LagBodyForceFcnData` + `registerLagBodyForceFunction` (slide 19)
“`Example_IBFE_TargetPoints2D`”	IBFE/01 (the example named in 2014 never shipped under that name)
`static double mu = 100` on the slides, 10 in the examples	`SHEAR_MODULUS` in the input; both values run (slide 16)
the 2018 overview’s four equation images	the theory repository’s chapters 4.1–4.3 (own derivations, own figures)

EXERCISES

1. Remove the dilational function from IBFE/01 (comment out its `registerPK1StressFunction`) and run `FULL` to $t = 0.5$. Measure the plate's length from `output.ex2` and explain the sign of the change from slide 9.
2. Replace the neo-Hookean pair by the compressible law of IBFE/10 with $\beta = 10\mu$ and $\beta = 100\mu$ (`MOTION = "HALF"`). Print J's range every 50 steps as IBFE/10 does. Which β is worth its time step?
3. Write the St. Venant–Kirchhoff law of ex7 for the plate with $\lambda = \mu$ and compare the tilt at $t = 1$ with the neo-Hookean 39°.
4. Make the plate's left half ten times stiffer than its right (test `s(0)` in both stress functions), run `HALF`, and explain the tilt you get from slide 16.
5. Add gravity (slide 21) with $\rho_s - \rho_f = 0.2$ and $g = 1$ to `MOTION = "FULL"` after the release; where is the centre at $t = 4$ compared with chapter 17's -0.936?
6. Make the tether of IBFE/01 a damped one as in ex5 ($\eta_s = 10^3$): register the velocity system, read it from `var_data`, and compare the overshoot at $t = 1$ with slide 26's 0.507.
7. Turn the plate's tether into a surface force on its long edges only (ex8's `side` test). Does the plate still follow the target, and what does its mesh look like at $t = 1$?

RULES OF THUMB

- A stress function returns $\mathbb{P}(\mathbb{F})$ and is called at every quadrature point every step: arithmetic only.
- Two parts for the neo-Hookean law, $\mu\mathbb{F}$ and $-\mu\mathbb{F}^{-T}$; the second makes the reference state stress-free, the fluid keeps the volume.
- Register stresses as many as you like, one body force per part, one surface force and one surface pressure per part; a body force is a `LagBodyForceFcnData`, never a `PK1StressFcnData`.
- $\mu/(\rho U^2)$: 4 folds, 40 bends, 400 barely moves, for a plate driven at U. Stiffer costs time step.
- κ_s: lag $= F_{\text{drag}}/(\kappa_s A)$; keep it below a grid cell.
- Set `F` on every path; test `s` (reference position) for "which part", `time` for "when", `FF` for "how much".
- Want the velocity or another field inside a function: `SystemData` in the `*FcnData`, read `var_data[i]`.
- Constants from the input, into `ModelData` statics or a `ctx` struct, before `initializeFEEquationSystems()`.

SUMMARY

1. The structure's elasticity is one or more C++ functions returning the first Piola–Kirchhoff stress from the deformation gradient; IBAMR integrates $-\mathbb{P} : \nabla\phi_k$ over every element with the Gauss rule you choose, solves for nodal forces, and spreads them from quadrature points.
2. The course's neo-Hookean model is $\mu\mathbb{F} - \mu\mathbb{F}^{-T}$ in two registered functions. The second term cancels the first in the reference state; it does not enforce incompressibility, the fluid does. The compressible variant adds $\beta \ln J$; St. Venant–Kirchhoff is $\mathbb{F}\mathbb{S}$ with Hooke's law in $\mathbb{E}$; all three are in the examples with their registrations.
3. Body forces (per reference volume) and surface forces (per reference area) have the same shape as stress functions plus, for surfaces, the normals and the side; the tether is the standard body force, gravity the next, a damped tether needs the velocity through `SystemData`. One body force per part, registered as a body force.
4. From runs: $\mu = 1,\ 10,\ 100$ fold, bend and hold the half-tethered plate; $\kappa_s = 10^4$ lags and overshoots where 10^6 follows. Both knobs trade fidelity against the time step.
5. The 2014 deck was right about registration and wrong about incompressibility; the 2020 deck inherited the second and added a registration that does not compile. Both are corrected here, with the theory left to the theory repository.

Chapter 20 adds the active fibre stress $J\,T\,\mathbb{F}\,\mathbf{f}_0 \otimes \mathbf{f}_0$ and runs the course's three fibre-tension examples (IBFE/06).

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recordings of the IBFE lectures (*Apply Active Tension*, which covered this chapter and the next) are listed in the front matter's video list (chapter 0.1). Nothing in them is reproduced here.

REFERENCES AND CREDITS

- B. E. Griffith and X. Luo, *Hybrid finite difference/finite element immersed boundary method*, Int. J. Numer. Meth. Biomed. Eng. 33 (2017) e2888 (the weak form, the quadrature-point spreading, the stress splitting).
- J. Bonet and R. D. Wood, *Nonlinear Continuum Mechanics for Finite Element Analysis*, 2nd ed., Cambridge University Press, 2008 (the 2014 deck's book link). G. A. Holzapfel, *Nonlinear Solid Mechanics*, Wiley, 2000 (hyperelastic laws for tissue).
- The theory repository: *Fluid Dynamics and IBM Theory*, chapters 4.1 *The mechanics, coupling and tradeoffs of IBFE* (30 slides), 4.2 *Kinematics, strain, stress and hyperelasticity* (83), 4.3 *Strong form, weak form and discrete coupling* (40). https://github.com/fairyflies9/Fluid-Dynamics-and-IBM-Theory
- IBAMR 0.19.0: `src/IB/FEMechanicsBase.cpp` (the force assembly and the registration functions), `include/ibamr/FEMechanicsBase.h`, `ibtk/include/ibtk/libmesh_utilities.h` (`SystemData`, `tensor_inverse_transpose`, `outer_product`); `examples/IBFE/explicit/ex2`, `ex4`, `ex5`, `ex7`, `ex8`. https://github.com/IBAMR/IBAMR
- The examples of this series: IBFE/01 (the runs of slides 16 and 26), IBFE/10 (the compressible law); `wsl/final/visit_ch19.sh`.
- Sources: Alex Hoover, *3-IBFE Tutorial: PK1 Stress and Body Forces* (UNC, 2014, edited 2016); *4-IBFE Lecture: Apply Active Tension*, slides 1–14 (2020; "adapted from Alex Hoover"); *2-IBFE Method: General Overview* (2018).